

C++ 面向对象编程：继承与多态详解

在C++中，**继承**和**多态**是实现面向对象编程（OOP）的两个核心概念。它们不仅提高了代码的复用性和可维护性，还增强了程序的灵活性和扩展性。本文将详细介绍继承与多态的相关知识，包括虚函数、纯虚函数，以及静态类型和动态类型的概念。通过表格、示例代码等多种展示方式，全面解析这些关键概念。

目录

- 继承 (Inheritance)
 - 继承的类型与权限管理
 - 公有继承 (Public Inheritance)
 - 保护继承 (Protected Inheritance)
 - 私有继承 (Private Inheritance)
 - 继承的基本语法与示例
- 多态 (Polymorphism)
 - 静态多态 (编译时多态)
 - 动态多态 (运行时多态)
- 虚函数与纯虚函数
 - 虚函数 (Virtual Function)
 - 纯虚函数 (Pure Virtual Function)
 - 虚函数与纯虚函数的区别
 - 为纯虚函数提供默认实现
- 静态类型与动态类型
- 知识点总结

继承 (Inheritance)

继承是面向对象编程中的一种机制，允许一个新类（子类）从一个或多个现有类（基类）获取属性和方法。通过继承，子类可以复用父类的代码，并在此基础上进行扩展或修改。

继承的类型与权限管理

C++支持三种继承类型：**公有继承**、**保护继承**和**私有继承**。每种继承方式通过不同的权限修饰符控制基类成员在派生类中的可访问性。

继承类型	基类公有成员	基类保护成员	基类私有成员	子类中基类成员的访问权限	适用场景
公有继承	公有	保护	无法访问	公有（公有成员）、保护（保护成员）	表示“是一个”关系
保护继承	保护	保护	无法访问	保护	限制子类成员对外部的可见性
私有继承	私有	私有	无法访问	私有	表示“不是一个”关系，通常用于实现组合

公有继承 (Public Inheritance)

- 定义**：使用 `public` 关键字进行继承。
- 权限控制**：
 - 基类的公有成员 → 子类的公有成员
 - 基类的保护成员 → 子类的保护成员
 - 基类的私有成员 → 子类不可访问
- 适用场景**：表示“**是一个**”的关系。例如，`Dog` 是 `Animal` 的一种。

示例代码：

```

class Animal {
public:
    void eat() {
        std::cout << "Animal eats." << std::endl;
    }
protected:
    void breathe() {
        std::cout << "Animal breathes." << std::endl;
    }
private:
    void sleep() {
        std::cout << "Animal sleeps." << std::endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        std::cout << "Dog barks." << std::endl;
    }

    void show() {
        eat();          // 可访问公有成员
        breathe();      // 可访问保护成员
        // sleep();     // 错误，无法访问私有成员
    }
};

```

保护继承 (Protected Inheritance)

- **定义：**使用 `protected` 关键字进行继承。
- **权限控制：**
 - 基类的公有成员 → 子类的保护成员
 - 基类的保护成员 → 子类的保护成员
 - 基类的私有成员 → 子类不可访问
- **适用场景：**希望派生类能访问基类的成员，但不希望外部代码访问基类的公有成员。

示例代码：

```

class Animal {
public:
    void eat() {
        std::cout << "Animal eats." << std::endl;
    }
protected:
    void breathe() {
        std::cout << "Animal breathes." << std::endl;
    }
private:
    void sleep() {
        std::cout << "Animal sleeps." << std::endl;
    }
};

class Dog : protected Animal {
public:
    void bark() {
        std::cout << "Dog barks." << std::endl;
    }

    void show() {
        eat();          // 作为保护成员
        breathe();      // 作为保护成员
        // sleep();     // 错误，无法访问私有成员
    }
};

```

私有继承 (Private Inheritance)

- **定义**：使用 `private` 关键字进行继承。
- **权限控制**：
 - 基类的公有成员 → 子类的私有成员
 - 基类的保护成员 → 子类的私有成员
 - 基类的私有成员 → 子类不可访问
- **适用场景**：表示“**不是一个**”的关系，通常用于实现类的内部组合。

示例代码：

```
class Animal {
public:
    void eat() {
        std::cout << "Animal eats." << std::endl;
    }
protected:
    void breathe() {
        std::cout << "Animal breathes." << std::endl;
    }
private:
    void sleep() {
        std::cout << "Animal sleeps." << std::endl;
    }
};

class Dog : private Animal {
public:
    void bark() {
        std::cout << "Dog barks." << std::endl;
    }

    void show() {
        eat();          // 作为私有成员
        breathe();      // 作为私有成员
        // sleep();     // 错误，无法访问私有成员
    }
};
```

继承的基本语法与示例

继承的基本语法使用 `:` 符号，后跟继承类型（`public`、`protected` 或 `private`）和基类名称。

基本语法：

```
class DerivedClass : access_specifier BaseClass {
    // 派生类成员
};
```

完整示例：

```
#include <iostream>
using namespace std;

class Base {
public:
    void display() {
        cout << "Base class display" << endl;
    }
protected:
    int protectedVar;
private:
    int privateVar;
};

class Derived : public Base {
public:
    void show() {
        display();           // 可访问公有成员
        protectedVar = 10; // 可访问保护成员
        // privateVar = 20; // 错误，无法访问私有成员
    }
};

int main() {
    Derived obj;
    obj.display(); // 可访问公有成员
    // obj.protectedVar = 5; // 错误，保护成员不可直接访问
    // obj.privateVar = 5;   // 错误，私有成员不可访问
    return 0;
}
```

多态 (Polymorphism)

多态是指不同类的对象对相同操作的响应不同。它使得相同的接口在不同的对象上可以有不同的实现。C++中的多态主要分为**静态多态**和**动态多态**。

静态多态 (编译时多态)

静态多态通过**函数重载**和**运算符重载**实现。编译器在编译时根据函数签名或运算符的类型决定使用哪个版本的函数。

示例代码：

```

#include <iostream>
using namespace std;

class Printer {
public:
    void print(int x) {
        cout << "Printing integer: " << x << endl;
    }

    void print(double x) {
        cout << "Printing double: " << x << endl;
    }

    void print(string x) {
        cout << "Printing string: " << x << endl;
    }
};

int main() {
    Printer p;
    p.print(10);           // 调用 print(int)
    p.print(3.14);         // 调用 print(double)
    p.print("Hello");      // 调用 print(string)
    return 0;
}

```

输出:

```

Printing integer: 10
Printing double: 3.14
Printing string: Hello

```

动态多态（运行时多态）

动态多态通过**虚函数**和基类指针或引用实现。在运行时，根据对象的实际类型决定调用哪个函数版本。

示例代码:

```

#include <iostream>
using namespace std;

class Base {
public:
    virtual void display() {
        cout << "Base class display" << endl;
    }
};

class Derived : public Base {
public:
    void display() override {
        cout << "Derived class display" << endl;
    }
};

int main() {
    Base* basePtr;
    Derived derivedObj;

    basePtr = &derivedObj;
    basePtr->display(); // 调用 Derived 的 display (输出 "Derived class display")

    return 0;
}

```

输出:

虚函数与纯虚函数

虚函数（Virtual Function）

定义：虚函数是基类中声明的一个成员函数，通过在函数声明前加上 `virtual` 关键字实现。它允许派生类重写该函数，实现动态绑定。

语法：

```
class Base {
public:
    virtual void show() {
        std::cout << "Base class show" << std::endl;
    }
};
```

使用场景：基类和派生类之间需要实现多态性，即派生类可以根据自身需求重写基类的虚函数。

示例代码：

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() { // 虚函数
        cout << "Animal speaks" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Woof!" << endl;
    }
};

class Cat : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Meow!" << endl;
    }
};

int main() {
    Animal* animal;
    Dog dog;
    Cat cat;

    animal = &dog;
    animal->speak(); // 输出 "Woof!"

    animal = &cat;
    animal->speak(); // 输出 "Meow!"

    return 0;
}
```

输出：

Woof!
Meow!

纯虚函数 (Pure Virtual Function)

定义： 纯虚函数是没有实现的虚函数，在基类中声明但不定义具体实现。它强制派生类必须提供该函数的实现，使得基类成为**抽象类**，无法实例化。

语法：

```
class Base {
public:
    virtual void show() = 0; // 纯虚函数
};
```

使用场景： 用于定义抽象基类，作为接口类，要求所有派生类实现特定的函数。

示例代码：

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // 纯虚函数
    virtual ~Shape() {}      // 虚析构函数
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing Rectangle" << endl;
    }
};

int main() {
    Shape* shape;

    Circle circle;
    Rectangle rectangle;

    shape = &circle;
    shape->draw(); // 输出 "Drawing Circle"

    shape = &rectangle;
    shape->draw(); // 输出 "Drawing Rectangle"

    return 0;
}
```

输出：

```
Drawing Circle
Drawing Rectangle
```

虚函数与纯虚函数的区别

特性	虚函数 (Virtual Function)	纯虚函数 (Pure Virtual Function)
是否有默认实现	有默认实现	无默认实现，必须被派生类实现
派生类的要求	派生类可以重写或不重写虚函数	派生类必须重写纯虚函数
是否可以实例化	可以实例化基类对象（若无其他纯虚函数）	不可以实例化基类对象

特性	虚函数 (Virtual Function)	纯虚函数 (Pure Virtual Function)
是否能作为接口	可以 (但通常有实现)	是, 纯虚函数通常用于定义接口

为纯虚函数提供默认实现

虽然纯虚函数通常没有实现，但可以在基类中提供一个默认实现。派生类可以选择重写该函数，或显式调用基类的默认实现。

示例代码：

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // 纯虚函数

    // 基类提供的默认实现
    void defaultDraw() {
        cout << "Drawing Shape (default implementation)" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }

    void drawWithDefault() {
        // 显式调用基类的默认实现
        defaultDraw();
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        // 使用基类的默认实现
        defaultDraw();
    }
};

int main() {
    Circle circle;
    Rectangle rectangle;

    // 使用派生类的实现
    circle.draw(); // 输出 "Drawing Circle"

    // 使用派生类显式调用基类的默认实现
    circle.drawWithDefault(); // 输出 "Drawing Shape (default implementation)"

    // 使用派生类的默认实现
    rectangle.draw(); // 输出 "Drawing Shape (default implementation)"

    return 0;
}
```

输出：

```
Drawing Circle
Drawing Shape (default implementation)
Drawing Shape (default implementation)
```

关键点：

- 纯虚函数**可以在基类中提供默认实现，但必须在基类外部定义。

- 派生类可以选择重写纯虚函数，或显式调用基类的默认实现。
- 通过这种方式，基类既能保持抽象性，又能为派生类提供通用的实现。

静态类型与动态类型

在C++中，对象有两种类型：**静态类型**和**动态类型**。

类型	定义	决定时机
静态类型	编译时确定，变量声明时指定的类型	编译时
动态类型	运行时确定，实际指向或引用的对象类型	运行时

示例代码：

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {
        cout << "Base class show" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived class show" << endl;
    }
};

int main() {
    Base baseObj;
    Derived derivedObj;

    Base* ptr;

    // 静态类型为 Base*，动态类型为 Derived
    ptr = &derivedObj;

    ptr->show(); // 输出 "Derived class show"

    return 0;
}
```

解释：

- **静态类型**：ptr 的静态类型是 Base*。
- **动态类型**：ptr 实际指向 Derived 对象，因此动态类型为 Derived。
- 调用 ptr->show() 时，因 show() 是虚函数，会根据动态类型决定调用 Derived 的实现，实现动态多态。

知识点总结

以下表格总结了继承、多态、虚函数和纯虚函数的关键知识点、用法及注意事项。

知识点	描述	用法与示例	注意事项
继承 (Inheritance)	允许创建一个新类从现有类获取属性和方法	class Derived : public Base { ... };	确保遵循“是一个”关系，合理选择继承类型
公有继承	基类公有成员 → 子类公有成员，保护成员 → 子类保护成员	class Derived : public Base { ... };	表示“是一个”关系，适用于大多数继承场景

知识点	描述	用法与示例	注意事项
保护继承	基类公有成员和保护成员均 → 子类保护成员	<code>class Derived : protected Base { ... };</code>	限制基类成员对外部的可见性 适用于特定场景
私有继承	基类公有成员和保护成员均 → 子类私有成员	<code>class Derived : private Base { ... };</code>	表示“不是一个”关系， 常用于实现类的内部组合
多态 (Polymorphism)	相同接口在不同对象上的不同实现	使用虚函数实现动态多态， 函数重载实现静态多态	动态多态需要基类指针或引用 虚函数是实现关键
静态多态	通过函数重载或运算符重载实现， 在编译时决定调用哪一个函数	<code>void print(int); void print(double);</code>	编译时决定，无法在运行时更改
动态多态	通过虚函数和基类指针或引用实现， 在运行时决定调用哪个函数	<code>virtual void show();</code>	需要虚函数， 运行时性能略低于静态多态
虚函数 (Virtual Function)	基类中声明的函数，通过 <code>virtual</code> 关键字实现动态绑定	<code>virtual void show() { ... }</code>	使用 <code>override</code> 关键字可增加代码的可读性和安全性
纯虚函数 (Pure Virtual Function)	基类中声明但不实现的虚函数， 强制派生类实现该函数	<code>virtual void draw() = 0;</code>	基类成为抽象类，无法实例化 派生类必须实现纯虚函数
为纯虚函数提供默认实现	在基类中为纯虚函数提供实现， 派生类可选择调用	在基类外部定义纯虚函数的实现， 派生类调用基类方法	需在基类外部定义， 实现方式不常见，需谨慎使用
静态类型	编译时确定，变量声明时的类型	<code>Base* ptr;</code>	决定编译时的接口与可访问成员
动态类型	运行时确定， 变量实际指向或引用的对象类型	<code>ptr = &derivedObj;</code>	决定运行时调用的函数实现， 依赖虚函数

关键知识点与注意事项

1. 选择合适的继承类型：
 - 公有继承适用于大多数场景，表示“是一个”关系。
 - 保护继承和私有继承用于更复杂的访问控制需求。
2. 理解多态的实现机制：
 - 静态多态通过函数重载和运算符重载实现，不涉及运行时决策。
 - 动态多态通过虚函数实现，依赖运行时的类型信息。
3. 正确使用虚函数和纯虚函数：
 - 虚函数允许派生类重写，实现动态多态。
 - 纯虚函数用于定义接口，强制派生类实现特定功能。
4. 管理静态类型与动态类型：
 - 静态类型决定了可访问的成员和接口。
 - 动态类型决定了实际调用的函数实现。
5. 虚析构函数：
 - 当基类有虚函数时，应将析构函数声明为虚析构函数，确保通过基类指针删除派生类对象时，能正确调用派生类的析构函数。

示例代码：

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {
        cout << "Base show" << endl;
    }

    virtual ~Base() { // 虚析构函数
        cout << "Base destructor" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived show" << endl;
    }

    ~Derived() {
        cout << "Derived destructor" << endl;
    }
};

int main() {
    Base* ptr = new Derived();
    ptr->show(); // 输出 "Derived show"
    delete ptr; // 正确调用 Derived 和 Base 的析构函数
    return 0;
}
```

输出：

```
Derived show
Derived destructor
Base destructor
```

注意事项：

- **虚析构函数**确保通过基类指针删除派生类对象时，能正确调用派生类的析构函数，避免资源泄漏。
- 使用 **override** 关键字增强代码的可读性和安全性，确保派生类正确重写基类的虚函数。

结论

通过本文的详细介绍，我们深入了解了C++中的继承与多态，包括继承的类型与权限管理、虚函数与纯虚函数的概念及其应用，以及静态类型与动态类型的关系。这些概念共同构成了C++面向对象编程的基础，掌握它们有助于编写高效、可维护且具有良好扩展性的代码。

关键点回顾

- **继承**允许类之间共享和复用代码，选择合适的继承类型至关重要。
- **多态**使得相同的接口可以在不同的对象上表现出不同的行为，分为静态多态和动态多态。
- **虚函数**实现了动态多态，通过基类指针或引用调用派生类的函数。
- **纯虚函数**定义了抽象接口，强制派生类实现特定功能，常用于接口类设计。
- **静态类型与动态类型**的概念帮助理解编译时与运行时的类型决策过程。

掌握这些知识点，将显著提升你在C++面向对象编程中的设计与实现能力。